

Выступление Джека Рашера, успешного технологического предпринимателя и ученого с богатым опытом работы в Bell Labs, посвящено критике подхода к разработке и программированию, который он называет «writing dead programs», что можно дословно перевести как «написание мертвого кода». Автор постепенно погружает зрителя в историю, начиная с давних времен, когда код писали еще на перфокартах (кстати, ограничение в 80 символов в строку в некоторых современных редакторах кода пошло именно отсюда). Именно с этой эпохой исторически к нам перешел подход написания «мертвого» кода, который Джек трактует следующим образом: такой код пишется заранее, затем компилируется и запускается, а по завершении выполнения программа «умирает». Автор считает, что идея разрабатывать продукты именно так до сих пор существует, несмотря на то что реальные серверы и веб-приложения могут работать дольше и постоянно обрабатывать события. В противовес такому подходу автор предлагает диаметрально противоположную идею – концепцию живых программ. Такие программы будут работать непрерывно, изменения в коде применяются без перезапуска, а разработчики могут исследовать систему в реальном времени. Джек Рашер считает, что не нужно создавать новые языки, которые будут решать эти проблемы, но стоит создавать новые среды программирования.

Идеи, описанные Рашером, актуальны и все чаще заметны в современной индустрии. Разработка сегодня действительно стремится уйти от «мертвых программ» – активно используется hot reload, позволяющий вносить изменения без остановки программы, интерактивные среды наподобие REPL, а также внедряется live debugging. Все это непосредственно влияет на продуктивность разработчика, поскольку сокращает feedback loop. Усиливается и интерес к DevEx (Developer Experience) – удобство инструментов и скорость разработки. Например, Jupyter Notebook часто используется как среда интерактивной разработки, React предоставляет разработчикам hot reload, а Node.js позволяет реализовывать долгоживущие процессы. Однако, как ни парадоксально, подход написания живых программ не универсален. Он сложен для больших систем и может ухудшать контроль над изменениями, так как в продакшене по-прежнему остаются важны стабильность, воспроизводимость и тестирование.

Мне кажется, что поднятая автором тема выходит за рамки языков программирования или каких-то конкретных инструментов. Как говорил сам Рашер, разработка – это не просто написание кода, это также и постоянное уточнение требований, которые могут непрерывно меняться, это поиск лучших решений, которые зачастую находятся путем долгих проверок и экспериментов. На мой взгляд, тяжело найти идеальный баланс между гибкостью и контролем. С одной стороны, живые системы дают нам скорость и маневренность для изменений. С другой, классический подход дарует нам предсказуемость и контроль, которые могут быть критически важны в некоторых

сферах. Не получится отказаться полностью от какой-то идеи, так или иначе их надо балансировать, чтобы достичь лучшего результата. Как вариант, гибкий подход можно использовать на этапе разработки, что позволит быстро получать обратную связь, а более консервативный, пришедший к нам давно подход к написанию кода мог бы использоваться в продакшене. Возвращаясь к одному из основных тезисов Рашера, создавать новые языки программирования ради решения конкретных идеологических подходов к разработке – не самоцель. Уже сегодня у нас есть огромное количество интерактивных сред разработки, ИИ-инструментов и сервисов для автоматизации разработки, которые позволяют нам привносить элементы интерактивности и «живого» взаимодействия с кодом – все то, что активно предлагал Джек Рашер.